2021204042 황인성
컴퓨터그래픽스 과제1

0.이전 버전 Three js 최신버전으로 migration

기본적으로 skeleton 코드와 거의 비슷한 방법으로 migration을 진행하였습니다. 차이점은 TweenMax를 최신 버전의 gsap 으로 교체하였고, 이에따라 추가적으로 importmap을 수정하였고, 쉐이더 및 light 설정도 약간의 차이가 있습니다.

```
<script type="importmap">
    {
        "imports": {
            "three": "./node_modules/three/build/three.module.js",
            "BufferGeometryUtils": "./node_modules/three/examples/jsm/utils/BufferGeometryUtils.js",
            "addons/": "./node_modules/three/examples/jsm/",
            "gsap": "./node_modules/gsap/index.js"
        }
    }
</script>
<script type="module" src="js/game.js"></script>
```

1.카메라 시점 변경 및 부드러운 카메라 이동

기본 aviator 코드 분석

updatePlane 함수는 게임 내 비행기와 카메라의 움직임을 제어합니다. targetY 라는 변수는 마우스의 y (높이) 값을 바탕으로 구해지는 변수이고, 비행기의 높이와 x(앞) 방향 회전 값을

```
airplane.mesh.position.y += (targetY - airplane.mesh.position.y) * deltaTime * game.planeMoveSensivity;
airplane.mesh.position.x += (targetX - airplane.mesh.position.x) * deltaTime * game.planeMoveSensivity;
airplane.mesh.rotation.z = (targetY - airplane.mesh.position.y) * deltaTime * game.planeRotXSensivity;
airplane.mesh.rotation.x = (airplane.mesh.position.y - targetY) * deltaTime * game.planeRotZSensivity;
```

해당 식으로 감쇠를 적용하여 마치 마우스로 실제 비행기를 조종하는 것처럼 구현되어있습니다. 이러한 기능을 그대로 가져가면서 시점 전환을 하는 것을 목표로 구현 목표를 잡았습니다.

문제 1. 카메라 높이 임의 설정

```
camera.position.y += (airplane.mesh.position.y - camera.position.y)*deltaTime*game.cameraSensivity;
```

문제는 해당 코드인데, 비행기의 높이에 따라 또 다시 감쇠를 적용해 카메라의 시점 높낮이를 조정하는 코드가 있습니다. 기본 3인칭 시점에서는 문제가 없지만, 새로 구현하려하는 1인칭 시점에서는 카메라가 비행기의 자식 좌표계가 되므로, 이 코드는 수정이 필요합니다.

```
if(!isFirstPerson){
  cam3y += (airplane.mesh.position.y - cam3y)*deltaTime*game.cameraSensivity;   // 3인칭 상태에서만 카메라가 비행기 높이에 천천히 맞춰서 이동
  if(!transitioning){
    camera.position.y = cam3y;
  }
}
```

해당 계산 값을 따로 변수로 저장하여 시점 전환 중에도 해당 값을 사용할 수 있게 처리하였습니다.

문제 2. rotation 순서 문제

변경전

변경후

또다른 문제로, 1인칭 모드를 보면 rotation.z 와 rotation.x 가 적용 될 때, rotation.x (roll) 가 먼저 적용되고, 그 후 rotation z (pitch) 가 적용되어,
비행기의 x 축이 월드 기준 x 축과 틀어지는 문제가 생기고 있었습니다. 이로인해 1인칭 시점에서 카메라 시점이 부자연스러움을 느꼈습니다.
이 문제를 해결하기 위해서 직감적으론 비행기의 z 축 회전은 월드 기준에서 처리하고, 그 후 x 축 회전은 로컬 기준으로 처리하는 아이디어를 떠올렸지만, 결과는 똑같았습니다.

```
airplane.mesh.rotation.order = "ZXY";
```

이후 로테이션의 순서를 바꿀 수 있는 코드가 있는걸 발견해서 이를 적용하여 문제를 해결하였습니다.

문제 3. 비행기의 roll 으로 인한 시점 변화

또한, 위 코드와 연관지어 비행기는 마우스 y 축 움직임에 따라 x 방향 회전이 나타나므로, 비행기를 부모로 하기 전(전환 애니메이션 중)과 후를 비교할 때, 갑작스러운 카메라 시점 변경이 발생할 수 있으므로, 이 또한 고려해야할 문제입니다.

해결 아이디어 (skeleton 코드에서는 시점 전환 새로운 함수를 선언했지만, 저는 updatePlane 함수를 수정하여 구현했습니다. 두 함수 다 loop 문 안에서 반복 실행되므로 구조상 차이는 없습니다.)

1.전환 애니메이션에 카메라의 x 방향(카메라 기준 z) 축 회전도 추가한다.

2.비행기의 부모를 추가하고, 그 부모에 카메라를 고정시킨다.

3.loop 문에서 강제로 카메라의 시점을 보정한다.

- 2번은 카메라가 비행기의 roll 회전을 무시할 수 있습니다.
- 3번이 가장 난이도는 쉽지만, loop에서 무한히 실행해야 하므로 리소스 낭비가 있다고 판단했습니다.

```
const targetQuat = new THREE.Quaternion();
airplane.mesh.getWorldQuaternion(targetQuat);
// 카메라가 비행기의 +X 방향을 바라보도록 보정
// 즉, 카메라의 -Z축을 비행기의 +X축으로 회전시켜야 하므로
// Y축을 기준으로 -90도 돌리면 Z축이 X축을 바라보게 됨
const correction = new THREE.Quaternion().setFromAxisAngle(
    new THREE.Vector3(0, 1, 0), -Math.PI / 2
);
// 비행기 회전 쿼터니언에 보정 회전 곱해줌
targetQuat.multiply(correction);
// Slerp로 회전 적용
camera.quaternion.slerp(targetQuat, Math.pow(t,2));
```

1번을 구현하기 위해서, 3->1인칭 전환 시 카메라 쿼터니언을 비행기 쿼터니언으로 slerp 하는 아이디어를 구상했습니다. 카메라의 앞은 -Z, 비행기의 앞은 +X 이므로 이를 고려해 y 방향으로 -90 회전하는 쿼터니언을 생성해 이를 slerp 하였습니다.

카메라 시점 전환의 부드러움을 위해 카메라의 위치(position)은 lerp와 Math.pow 함수, 회전(rotation)은 slerp를 이용하였습니다. 3인칭 -> 1인칭, 1인칭 -> 3인칭 둘 다 같은 방식을 사용했습니다.

```
if (t >= 1) {
    transitioning = false;
    //airplane.mesh.add(camera);
    //camera.position.set(-650, 10, 0);
    airplane.mesh.updateMatrix();
    let T_A = airplane.mesh.matrix.clone();//  new THREE.Matrix4();
    camera.updateMatrix();
    let T_C0 = camera.matrix.clone();//  new THREE.Matrix4();
    airplane.mesh.add(camera);
    const inverseT_A = new THREE.Matrix4();
    inverseT_A.copy(T_A).invert();
    let T_C = new THREE.Matrix4();
    T_C.multiplyMatrices(inverseT_A, T_C0);
    camera.matrix.copy(T_C);
    camera.matrixAutoUpdate = false;
}
```

시점 전환 후 카메라 비행기 고정은 처음에는 비행기의 뒤 위치를 따로 저장하고 그 위치로 이동하는 방식을 사용했지만, 교수님께서 알려주신 방식으로 다시 수정하였습니다.

카메라 비행기 고정 해제는 이와 다르게, 고정되었을 때의 카메라의 월드 기준 트랜스폼이 자동으로 고정 해제 후에도 유지 되도록 three.js 가 처리합니다.

문제 4. 1인칭 -> 3인칭 문제

```
else {    // 3인칭 전환
  const targetDir = new THREE.Vector3(0, 0, -1); // 기본 3인칭 카메라 방향
  const up = new THREE.Vector3(0, 1, 0); // 기본 업 벡터
  // 카메라의 현재 위치 기준으로 목표 방향 생성
  const targetMatrix = new THREE.Matrix4().lookAt(
      camera.position.clone(),                          // from
      camera.position.clone().add(targetDir),           // to
      up                                                // up
  );
  // 목표 쿼터니언 생성
  const targetQuat = new THREE.Quaternion().setFromRotationMatrix(targetMatrix);
  // slerp 적용
  camera.quaternion.slerp(targetQuat, Math.pow(t,1));

  const targetPosition = new THREE.Vector3(
    airplane.mesh.position.x + 100,
    cam3y,
    200
  );
  camera.position.lerp(targetPosition,Math.pow(t,2)); // 위치 보간 (LERP)
```

간단하게 구현한 1->3인칭 코드입니다.

사소할 수도 있지만, 먼저 코딩한 3->1인칭 전환은 정지된 점에서 실시간으로 움직이는 비행기를 목표로 잡고 계산하는 과정으로 진행했는데, 1->3인칭 전환은 그 반대이므로,

이전에 코딩한 3->1인칭 코드를 응용하면 더 자연스럽게 시점 전환이 가능할 것이라고 생각했습니다.

```
//1인칭 시점에서 점점 3인칭 시점으로 변화
// 3 -> 1 코드에서 t 값을 반대로 감소시켜서 구현
// 1인칭 시점에서의 카메라 움직임을 lepr, slerp 로 서서히 감소
const targetPosition2 = airplane.mesh.localToWorld(new THREE.Vector3(-650, 10, 0));
camera.position.lerp(targetPosition2, 1-Math.pow(t,0.1));

const targetQuat2 = new THREE.Quaternion();
airplane.mesh.getWorldQuaternion(targetQuat2);
const correction = new THREE.Quaternion().setFromAxisAngle(
    new THREE.Vector3(0, 1, 0), -Math.PI / 2
);
targetQuat2.multiply(correction);
camera.quaternion.slerp(targetQuat2, 1-Math.pow(t,0.1));
```

그래서 기존 코드 아래에 t 값을 반대로 감소시켜서 1인칭 시점의 카메라 움직임을 서서히 감소시키는 코드를 추가하였습니다.

기존 코드는 1인칭 시점에서 전환을 시작한 순간의 transform에서 3인칭 시점으로 이동한다면,

이 코드를 추가하면서 1인칭 시점에서 서서히 3인칭 시점으로 전환되도록 구현하였습니다.

2.Sea geometry 끊어지는 부분 처리

gpt와 오랜 시간 문제를 해결하기 위해 고민했었습니다. 그에 비해 해결방법은 매우 간단했습니다.

```
Sea = function(){
  var geom = new THREE.CylinderGeometry(game.seaRadius,game.seaRadius,game.seaLength,40,10);
  geom.applyMatrix(new THREE.Matrix4().makeRotationX(-Math.PI/2));
  geom.mergeVertices();
  var l = geom.vertices.length;
  //console.log(l);
```

이전 버전의 aviator 코드입니다. 이 코드에서는 geometry가 끊어지는 문제가 발생하지 않습니다. 디버깅을 위해 vertices 의 개수를 출력해보면 **442** 라는 값이 나옵니다.

```
function Sea() {
  var geom = new THREE.CylinderGeometry(game.seaRadius, game.seaRadius, game.seaLength, 40, 10);
  geom.applyMatrix4(new THREE.Matrix4().makeRotationX(-Math.PI / 2));
  geom = BufferGeometryUtils.mergeVertices(geom,2.1);
  //geom.computeVertexNormals(); // 병합 후 노멀 재계산
  //geom.attributes.needsUpdate=true;
  var l = geom.attributes.position.count;
  //console.log(l);     //병합 오차 0(기본값)~2 : 533, 2 초과 : 442, 이전버전 : 442
```

최신 버전의 three js에서는 BufferGeometryUtils.mergeVertices를 사용해야합니다.

이전 버전의 mergeVertices와 다르게, 따로 추가 인자 없이 geom 만 넣을 경우, 이 함수는 병합할 vertices의 허용 오차(tolerance)를 매우 낮게 적용한다고 합니다.

이 상태에서 vertices의 개수를 출력하면 무려 **533**이라는 값이 나옵니다.

이후 tolerance가 2보다 클 경우, 이전 버전과 동일하게 정상적으로 **442**라는 값이 출력되고, geometry가 끊어지는 문제가 해결되었습니다.

처음에는 이 함수에 geom 말고 다른 인자를 넣을 수 있는지 몰랐지만, 해당 함수에서 문제가 있는 것 같은 느낌이 들어서 찾아보게 되었습니다.

추가

```
//스페이스바 감지 리스너
document.addEventListener("keydown", (e) => {
  if (e.code === "Space") {
    isFirstPerson = !isFirstPerson;
    transitioning = true;
    transitionStartTime = performance.now();
    if(!isFirstPerson){
      airplane.mesh.remove(camera); // 비행기에서 분리
      camera.position.copy(airplane.mesh.localToWorld(new THREE.Vector3(-650, 100, 0)));
    }
  }
});
/* 기존 updatePlane…
function showReplay(){…
```

리스너는 loop 밖에 있어도 영구적으로 작동합니다.